Probabilistic Inductive Miner: Balancing Accuracy, Simplicity and Trust in Process Discovery

Suhas Nandihalli Umesh (Student 1)[1] and Rohit Suryaa Saravanan (Student 2)[1]

Abstract: Process discovery turns an event log into a process model, judged along dimensions that pull against each other: fitness, precision, simplicity, and the correctness condition of soundness. For two decades no algorithm guaranteed a sound and fitting model in finite time. The Inductive Miner framework of Leemans, Fahland and van der Aalst closed that gap in 2013 by restricting the output to block-structured process trees, but it reads the directly-follows graph as a set of binary facts, which makes it fragile on logs containing deviations. This report explains how the Probabilistic Inductive Miner (PIM) of Brons, Scheepens and Fahland repairs that fragility while preserving soundness. The first half, written by Suhas Nandihalli Umesh, reconstructs the 2013 framework, its guarantees, its polynomial instantiation and its rediscoverability result, isolates the assumption that breaks on real data, states the three requirements PIM was built to satisfy, and assesses its published evaluation. The second half, continued by Rohit Suryaa Saravanan, develops the probabilistic machinery: the 2014 incomplete-log paper's activity relation lattice, a worked computation tracing the pair (a, g) from its observed exclusive choice relation to $p_{\times}(a, g) = 0.60$, its residual sequence probability $p_{\rightarrow}(a, g) \approx 0.07$ and an aggregate root cut score of about 0.64, then PIM's own relation scores and correction terms, and an independent worked example computed from the same definitions. The report concludes that PIM does not dominate all dimensions, and that its evaluation supports a narrower claim: inside a framework that still guarantees soundness, it reaches a region of the accuracy and simplicity trade-off that industrial analysts were shown to prefer.

Keywords: Process Mining, Process Discovery, Inductive Miner, Process Trees, Soundness, Conformance, Event Logs

1 Introduction

Contemporary information systems record every step of the business processes they support as an event log: a collection of traces, each the ordered sequence of activities executed for one case [Aa16]. Turning such a log into a model of the process behind it is process discovery, and in industrial practice everything else rests on it: the model is what compresses thousands of traces into something an analyst can look at, and it is against that model that deviations become visible as deviations rather than as noise [BSF21].

The task is difficult because a log is only a sample. It shows some of the possible behaviour, rarely all of it, and in practice it also shows behaviour the process was never meant to produce. A discovered model is therefore judged along quality dimensions that pull against each other [BDA14]: fitness, how much observed behaviour the model replays; precision, how much unobserved behaviour it allows; generalisation, whether it accepts future behaviour absent

[1] Hochschule Fulda, Master Seminar: Algorithms in Process Mining, Supervised by Prof. Dr. Norbert Ketterer, Fulda, Germany,

from the log; and simplicity, how readable it is. Above these sits a correctness condition rather than a trade-off. A sound model has no deadlocks, no dead parts, and can always reach a proper end state [Aa16]; an unsound one can usually be discarded without consulting the log at all.

By 2013 the field had produced a long list of discovery algorithms and none guaranteed the combination that matters. Ordering-relation algorithms such as α rediscover small model classes but guarantee neither fitness nor soundness; region-based and ILP-based algorithms guarantee fitness and can still return unsound models; frequency-based algorithms tolerate noise and guarantee neither; genetic algorithms reach any target given unlimited time, which is no guarantee in finite time [LFA13]. Leemans, Fahland and van der Aalst broke the deadlock by shrinking the search space to block-structured models. Since every block-structured process tree translates into a sound workflow net, soundness becomes a property of the representation rather than something to prove, leaving only fitness and termination to establish.

Closing the theoretical gap did not close the industrial one. Practitioners kept reaching for directly-follows graphs, and the reason usually offered is that nothing in them has to be learned: an arrow means one activity came after another. Past a certain size a formal notation stops being readable, whereas a graph of arrows stays legible. The catch is that legibility here is bought by omitting concurrency, and a notation that cannot say two things happened in parallel will report throughput times and deviation counts that are simply wrong [BSF21]. Among the notations that can express concurrency, none was satisfactory on every count. Split Miner scores well on fitness and precision but produces tangled models and occasionally unsound ones; the Inductive Miner is sound and simple and fits well, and pays for it in precision [Au19; BSF21]. Brons, Scheepens and Fahland answered this with the Probabilistic Inductive Miner (PIM), an instantiation of the same 2013 framework whose design choices came from computational requirements and a user study with industrial experts rather than from a quality metric [BSF21].

Structure of this report and division of work. This report is written by two authors as one continuous argument. Sections 2 to 7, by Suhas Nandihalli Umesh, cover the framework side: Section 3 reconstructs the first foundational paper [LFA13], its two guarantees, the polynomial algorithm B', a worked run of it, its rediscoverability result, and an own example showing what a single deviating trace does to it; Section 4 states the three requirements PIM was built to satisfy and why IMf satisfies only part of them; Section 5 presents PIM as local substitutions inside an otherwise unchanged framework; and Section 6 assesses the published evaluation. Sections 8 to 13, by Rohit Suryaa Saravanan, develop the probabilistic machinery the first half only names.

2 Preliminaries

Logs and graphs. Let Σ be a finite set of activity names. A trace $\sigma \in \Sigma^*$ is a finite sequence of activity names observed for one process execution, and an event log $L \in \mathcal{B}(\Sigma^*)$ is a multiset of traces [BSF21]. We write $|L|$ for the number of traces and $\|L\|$ for the total number of events in L.

The directly-follows graph $\mapsto(L)$ has the activities as nodes and an edge $a \mapsto b$ if and only if b directly follows a in some trace of L; the indirectly-follows graph $\mapsto^*(L)$ has an edge $a \mapsto^* b$ if and only if b strictly indirectly follows a. The frequencies $|a \mapsto b|$ and $|a \mapsto^* b|$ count the occurrences of b that directly and indirectly follow some a. The example in [BSF21] is worth repeating, since the distinction is easy to get wrong: in $\langle a, a, b, c, b, b, a, b\rangle$ we have $|a \mapsto b| = 2$, the first and fourth b, while $|a \mapsto^* b| = 4$, all four. Finally, $\mathit{Start}(L)$ and $\mathit{End}(L)$ denote the first and last activities of the traces.

This graph is what almost every algorithm discussed here operates on, and also what commercial tools display to users. It is cheap and easy to read, but an edge in both directions is ambiguous between genuine concurrency and a short loop, which is why a directly-follows graph systematically misreports throughput times and deviations [Aa22; BSF21].

Process trees. A process tree is an abstract representation of a sound, block-structured workflow net [BSF21; LFA13]: a tree in which every leaf is an activity $a \in \Sigma$ or the silent activity τ and every non-leaf node is one of four operators. Exclusive choice $\times$ executes exactly one child; sequence $\rightarrow$ executes all children in order; interleaved parallelism $\wedge$ executes them in any interleaving; and loop $\circlearrowleft$ takes its first child as the body, repeatable after executing any of the other redo children.

Semantics are defined recursively by giving each operator $\oplus$ a language join function $\oplus_l$ combining its children's languages. What carries the construction is structural rather than semantic: each operator translates directly into a sound, block-structured workflow Petri net, so every process tree is sound by construction, and a fifth operator could be added by showing only that its own translation is sound in isolation [LFA13]. The flower model $\circlearrowleft(\tau, a_1, \ldots, a_m)$ fits every log over Σ and carries no information, which makes it the natural fallback when no structure is found.

3 Foundational Paper 1: The Inductive Miner Framework

3.1 Divide and conquer with a termination counter

The framework B, reproduced as Algorithm 1, discovers process trees by divide and conquer [LFA13]. Given L it looks for a split into sublogs $L_1, \ldots, L_n$ such that recombining them with some operator $\oplus$ reproduces at least L, then recurses on each sublog. Two base cases end

the recursion: a log holding only the empty trace yields τ, and a log holding a single activity yields that activity as a leaf. When neither applies and no split is found, the framework returns the flower model, the honest answer that the log shows no exploitable structure.

Which split to take is deliberately not part of the framework. It is delegated to a function $select(L)$ returning tuples $(\oplus, ((L_1, \varphi_1), \ldots, (L_n, \varphi_n)))$. That delegation is what makes this a framework rather than an algorithm, and the reason PIM can later replace cut selection outright without disturbing anything around it.

The second parameter φ is easy to overlook and is what makes the construction terminate. Insisting that every sublog be strictly smaller than its parent would be too strong, because models containing silent branches can only be rediscovered if a split may leave the log at its original size. The framework therefore allows size-preserving splits but attaches a counter that must strictly decrease whenever one is taken, the second condition of Definition 1. Intuitively φ bounds the number of invisible branches the model may contain, and the paper gives it a concrete value: the sum of the width and depth of the model to be rediscovered.

Definition 1 (valid log division, after [LFA13])**.** Every tuple $(\oplus, ((L_1, \varphi_1), \ldots, (L_n, \varphi_n)))$ returned by $select(L)$ must satisfy, for all i: $L \subseteq \oplus_l(L_1, \ldots, L_n)$; $\|L_i\| + \varphi_i < \|L\| + \varphi$; $\|L_i\| \leq \|L\|$; $\varphi_i \leq \varphi$; $\Sigma(L_i) \subseteq \Sigma(L)$; $\oplus$ is one of the available operators; and $n \leq \|L\| + \varphi$.

Algorithm 1 The framework $B_{select}(L, \varphi)$, after [LFA13]

if $L = \{\epsilon\}$ **then**
  $base \leftarrow \{\tau\}$
else if $L = \{\langle a \rangle\}$ for some $a \in \Sigma$ **then**
  $base \leftarrow \{a\}$
else
  $base \leftarrow \emptyset$
end if
$P \leftarrow select(L)$
if $P = \emptyset$ **then**
  return *base* if $base \neq \emptyset$, else the flower model $\circlearrowleft (\tau, a_1, \ldots, a_m)$
end if
return $\{\oplus(M_1, \ldots, M_n) \mid (\oplus, ((L_1, \varphi_1), \ldots, (L_n, \varphi_n))) \in P \wedge \forall i : M_i \in B(L_i, \varphi_i)\} \cup base$

3.2 The two guarantees

Two theorems hold for the framework itself, whatever *select* does, and they are the reason the framework is worth having at all.

Termination follows from Definition 1: $\|L\| + \varphi$ strictly decreases in every recursive call, the number of sublogs per call is bounded by $\|L\| + \varphi$, and the alphabet never grows, so the recursion tree is finite. Fitness follows by induction on $\|L\| + \varphi$. Base cases and the flower model fit trivially; otherwise the induction hypothesis gives $L_i \subseteq \mathcal{L}(M_i)$, monotonicity of

the language join functions lifts this to $\oplus_l(L_1, \ldots, L_n) \subseteq \mathcal{L}(M)$, and the first condition of Definition 1 supplies $L \subseteq \oplus_l(L_1, \ldots, L_n)$.

Soundness needs no proof at discovery time. Every output is a process tree and every process tree translates into a sound workflow net. This is the decisive move of the 2013 paper: soundness is obtained not by checking or repairing the output but by choosing a representation in which unsound models cannot be written down. The price is representational bias, since processes that are not block-structured can only be approximated, and that price returns in Section 6.

3.3 Algorithm B': reading cuts from the graph

The framework as stated is not implementable, because constructing the full set of valid log divisions is infeasible for realistic logs. The algorithm B' instantiates it by computing exactly one division directly from the directly-follows graph, with φ fixed at 0.

Each operator leaves a characteristic footprint in $\mapsto (L)$. An n-ary cut is a partition of the nodes into disjoint sets $\Sigma_1, \ldots, \Sigma_n$, and each operator corresponds to a condition on the edges running between them, illustrated in Fig. 1: no edge between any two sets for exclusive choice; an ordering in which for $i < j$ a path runs from every node of Σ_i to every node of Σ_j and never the reverse, for sequence; a start and an end node in every set together with connections in both directions between all sets, for parallelism; and, for loops, Σ_1 holding all start and end nodes, mutually disconnected redo sets, and every edge between body and redo set leaving an end node of the body or reaching a start node of it.

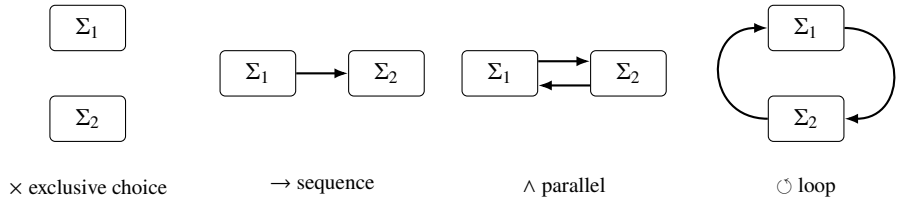


Fig. 1: The four operator footprints in the directly-follows graph. Absence of edges characterises exclusive choice, one-directional reachability characterises sequence, edges in both directions characterise parallelism, and re-entry restricted to the start and end activities of the body characterises a loop. Own figure, after the cut definitions in [LFA13].

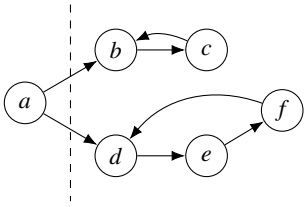
Two properties carry the practical weight. All four cuts reduce to standard graph computations, namely connected components for exclusive choice, collapsed strongly connected components for sequence, connected components of the inverted graph for parallelism, and connected components after removing the start and end activities for loops. Each is polynomial, and since the recursion depth is bounded by the number of activities, B' is polynomial overall. The cuts are tried in the fixed order $\times$, $\rightarrow$, $\wedge$, $\circlearrowleft$, taking a nontrivial maximal cut of the first applicable type; on a sufficiently complete log the four are mutually

exclusive, so the ordering simplifies implementation and proofs without being semantically necessary.

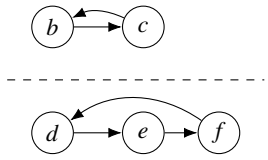
3.4 A worked run of the algorithm

Once a cut is identified the log is split accordingly: exclusive choice moves whole traces into the sublog whose activity set they belong to, sequence cuts each trace at the boundary, parallelism projects each trace onto each activity set, and loop splits each trace into body and redo subtraces.

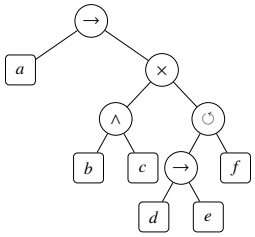
It is worth running this through once. Take the log $L = \{\langle a, b, c\rangle, \langle a, c, b\rangle, \langle a, d, e\rangle, \langle a, d, e, f, d, e\rangle\}$ of [LFA13], whose graph is Fig. 2(a). Activity a has no incoming edges and no path leads back to it, so the maximal sequence cut $\{a\} \mid \{b, c, d, e, f\}$ is taken. The right sublog, Fig. 2(b), falls into two connected components, giving the exclusive choice cut $\{b, c\} \mid \{d, e, f\}$. On $\{b, c\}$ the edges run both ways and both activities are start and end activities, the parallel condition, so $\{b\} \mid \{c\}$ is taken. On $\{d, e, f\}$ the start and end activities are d and e, and removing them leaves f isolated, the loop condition, so $\{d, e\} \mid \{f\}$ is taken and one further sequence cut splits $\{d, e\}$. The result is Fig. 2(c).



(a) $\mapsto (L)$, sequence cut



(b) $\mapsto (L_2)$, exclusive choice cut



(c) the resulting process tree

Fig. 2: Own figure. (a) the directly-follows graph of the running example log of [LFA13] and (b) of the sublog produced by the first split, with the identified cuts drawn as dashed lines; (c) the process tree $\rightarrow (a, \times(\wedge(b, c), \circlearrowleft (\rightarrow (d, e), f)))$ that B' returns.

3.5 Rediscoverability and what the log must contain

Fitness and soundness grade a model against the log. Rediscoverability asks something harder: if the log was produced by a real system M, does the algorithm return M again? The value of the 2013 paper is that it makes the requirement on the log explicit, and weak. For the framework that requirement is activity-completeness, every leaf of M occurring in L at least once, which together with a φ as large as the sum of the width and depth of M places every process tree in the set returned by B.

For B' the requirement is stronger, because B' sees only the directly-follows graph. A log is directly-follows complete with respect to M if every directly-follows pair and every start and end activity of M appears in it, which is far weaker than demanding every trace: a model with ten parallel activities admits $10! = 3{,}628{,}800$ interleavings, and such a log needs only enough of them to exhibit every ordered pair.

Under three model restrictions close to those of the α algorithm, namely no duplicate activities, disjoint start and end sets of a loop body, and no silent leaves, B' provably rediscovers the language of M. The proof reduces every tree to a normal form by flattening nested nodes with the same operator, then shows that a directly-follows complete log forces exactly one operator and exactly one split at every recursion step, so the original tree is rebuilt inevitably. The tree returned is moreover the smallest, most precise and most general one with that language among the trees satisfying the restrictions.

3.6 The assumption that does not survive real data

Everything above rests on a single assumption: the directly-follows graph is read as a set of binary facts. An edge is either present or absent, the cut conditions speak only of presence and absence, and B' computes no frequency anywhere.

On real logs that assumption breaks, and it is worth demonstrating how rather than asserting it. Consider a clean log $L_c = [\langle a, b, c, d\rangle^{50}]$, whose graph is the chain $a \mapsto b \mapsto c \mapsto d$ of Fig. 3(a). The algorithm finds no exclusive choice cut, since the graph is connected, then the maximal sequence cut $\{a\} \mid \{b\} \mid \{c\} \mid \{d\}$, returning $\rightarrow (a, b, c, d)$, which is exactly the process that generated the log.

Now add one deviating trace $\langle a, c, b, d\rangle$, in which b and c were recorded in the wrong order, giving $L_d = L_c + [\langle a, c, b, d\rangle]$. Three edges appear, $a \mapsto c$, $c \mapsto b$ and $b \mapsto d$, drawn dashed in Fig. 3(b). There is still no exclusive choice cut, but b and c now lie on a cycle, since both $b \mapsto c$ and $c \mapsto b$ are present, so they collapse into one strongly connected component and the maximal sequence cut becomes $\{a\} \mid \{b, c\} \mid \{d\}$. Recursing on the middle sublog $[\langle b, c\rangle^{50}, \langle c, b\rangle]$, the algorithm finds edges in both directions and both activities in *Start* and *End*, which is precisely the parallel condition, and returns $\wedge(b, c)$. The discovered model is $\rightarrow (a, \wedge(b, c), d)$.



(a) $\mapsto (L_{\mathrm{c}})$ (b) $\mapsto (L_{\mathrm{d}})$, deviation dashed

Fig. 3: Own example: one deviating trace out of fifty-one adds three edges to the directly-follows graph, which is enough to turn the sequence $\rightarrow (a, b, c, d)$ into $\rightarrow (a, \wedge(b, c), d)$.

The consequence is not a small numerical shift. One trace out of fifty-one has turned a strict ordering into a permanent parallel block, and the model now admits $\langle a, c, b, d\rangle$ as normal behaviour rather than as a deviation: precision drops and the deviation becomes invisible as such. The same mechanism runs the other way too, since when the added edges leave no cut of any type applicable the framework has nothing to return but the flower model, which fits perfectly and says nothing [BSF21; LFA13].

Incompleteness produces the mirror image of the defect. A pair that stands in sequence in the real system but is never observed adjacent is read as an exclusive choice, and once that relation has flipped, correct evidence elsewhere in the graph cannot pull it back, because nothing in B' weighs evidence at all. The authors of [LFA13] name the remedy themselves in their future work, proposing to filter the directly-follows relation before the graph is constructed. The algorithms discussed below are successive attempts at that.

4 The Requirements and the Limits of IMf

4.1 Three requirements derived from analysts

PIM is unusual in that its design constraints were elicited before the algorithm was designed, rather than chosen to match a benchmark metric [BSF21]. Two sources produced three requirements.

The first requirement falls out of a runtime budget. Simplicity is bought from fitness, since the models of a log lie on a pareto front [BDA14], so an algorithm aiming at simple models has to be able to put the omitted behaviour back on screen on demand. Alignments do that, and they are far too slow to sit inside an interactive loop. Their visual approximation is fast enough, but only under a condition: it runs in linear time provided the model is block-structured and its gateways are limited to XOR and AND. Those are also the models analysts find easiest to read. Hence **R0**, that the discovered model be block-structured with XOR and AND gateways only [BSF21]. Note what R0 is not: it is not a new demand, but the bias the 2013 framework already imposes, which is exactly why that framework was chosen as the starting point.

The second source is a Delphi user study, and its shape matters for how much weight R1 and R2 can carry. The panel was six expert process mining analysts. What they were shown came from three real-life event logs, cut into nine fragments; for every fragment the authors hand-built somewhere between two and eight competing block-structured models, deliberately varied in structural complexity, in fitness and in precision. Each analyst said which model best represented the fragment and why, and a second round consolidated both the choices and the reasons [BSF21]. Two requirements came out of it. **R1** asks for a parameter controlling model complexity, so that an analyst can dial detail up or down and pay for it in fitness or precision. **R2** asks that a structure appear only where the data supports it: parallelism, loops and skipped activities should surface only when they occur often enough that the analyst has no reason to doubt the choice.

R1 and R2 are exactly the two requirements the 2013 framework does not address. It exposes no parameter of any kind, and its notion of evidence is binary presence in the graph, as Section 3.6 showed.

4.2 Why IMf satisfies only part

One instantiation had already attempted the repair. IM infrequent (IMf) works downward: it proposes a partition of Σ and tests it against $\times$, $\rightarrow$, $\wedge$ and $\circlearrowleft$ in turn [BSF21; LFA14a]. To keep deviations from spoiling that test, it thins $\mapsto$ (L) first, discarding any edge that looks weak beside the busiest edge leaving the same activity. In symbols, $a \mapsto b$ goes if $|a \mapsto b| < f \cdot \max_{x \in \Sigma} |a \mapsto x|$. Should the thinned graph still admit no cut, the flower model is returned.

This threshold is local, with two consequences. The first is uniformity. Because each edge is compared only against the strongest edge leaving the same activity, two edges of identical absolute frequency can receive opposite verdicts depending on which activity they leave. The PIM paper illustrates this on its own running example, where $c \mapsto g$ and $g \mapsto c$ have the same frequency and are only partially filtered, and observes that this impairs IMf's ability to control model complexity uniformly, weakening R1 [BSF21].

The second is more serious. Searching downward while filtering locally means the search can walk past the cut that the data most supports, and it takes only slightly more deviation than the filter was tuned for to make that happen. On real logs the symptom is low precision [Au19; BSF21], which is the measurable form of the mechanism Section 3.6 demonstrated: a handful of extra edges is enough to push the algorithm onto a weaker structure, or off structure altogether.

The gap is now locatable. IMf has the right target, real logs with deviations, and the wrong mechanism, a local threshold. What is needed instead is a way to weigh all the evidence in the graph against itself, so that a handful of misread pairs is outvoted rather than decisive. That mechanism is what the second foundational paper supplies.

5 PIM Inside the Framework: What Is Kept and What Is Replaced

5.1 PIM as an instantiation

PIM instantiates the 2013 framework rather than replacing it. Its pseudocode, Algorithm 2, is the same recursion of BaseCase, FindCut and SplitLog with one step inserted and one reimplemented [BSF21]. Tab. 1 sets out the correspondence, the question the first half of this report is organised around.

Algorithm 2 PIM(L, f), after [BSF21]

```
bc ← BaseCase(L)
if bc ≠ ⊥ then
    return bc(L)
else
    ↦ (L)^f, ↦* (L)^f ← Filtering(↦ (L), ↦* (L), f)
    ⊕(Σ1, Σ2) ← FindCut(↦ (L)^f, ↦* (L)^f)
    L1, L2 ← SplitLog(L, ⊕(Σ1, Σ2))
    return ⊕(PIM(L1, f), PIM(L2, f))
end if
```

Tab. 1: What PIM inherits from the 2013 framework and what it substitutes.

Element	Status	Comment
BaseCase / FindCut / SplitLog recursion	kept	control flow unchanged
Process trees as output	kept	soundness by construction; satisfies R0
Operators $\times, \rightarrow, \wedge, \circlearrowleft$	kept	the same four cut types
Cut detection from edge presence	replaced	frequency-based pairwise scores
Cut choice by first matching footprint	replaced	highest corrected aggregate score
No filtering step	added	global percentile filtering; satisfies R1
Flower model fallback	dropped	a cut is always returned, possibly with a low score

5.2 Percentile filtering

Filtering sorts all edges in $\mapsto (L)$ and $\mapsto^* (L)$ by frequency, retains only the top f percent, discards the rest and removes nodes that have become disconnected [BSF21]. What separates this from IMf is the scope of the comparison. IMf weighs an edge against the strongest edge leaving the same activity and leaves $\mapsto^* (L)$ unfiltered, so two edges of identical frequency can receive opposite verdicts; PIM weighs every edge in both graphs against every other, so behaviour is removed uniformly, f behaves monotonically, and the parameter becomes a usable complexity dial, satisfying R1. The filter also does what the 2013 algorithm could not: it removes the deviating edges that would otherwise distort every

cut, before cut detection sees the graph. On the example of Section 3.6 it can remove all three added edges if its edge-frequency cutoff excludes them; that cutoff depends on where the edges fall in the global ranking, not on the ratio of deviating traces to clean ones.

5.3 Splitting, base cases and cost

SplitLog follows IMf's method, filtering from each sublog the events that deviate from the chosen cut, since the cut need not fit the log perfectly. Filtering may leave empty traces, denoting that the sublog's behaviour can also be skipped [BSF21], and PIM handles those differently from IMf in a direct application of R2. IMf filters the empty traces from L and returns $\times(\tau, \text{IMf}(L \setminus \{\langle\rangle\}))$ the moment their count reaches $|L| \cdot f$. Neither side of that threshold is comfortable: below it the evidence that skipping occurred is discarded, above it silent branches accumulate throughout the tree and precision falls. PIM postpones the decision instead. As long as the number n of empty traces stays at or below $0.5 \cdot |L|$ they are retained but left out of $\mapsto$, $\mapsto^*$ and the repetition ratio; the $\times(\tau, \ldots)$ appears only once n passes that halfway mark, which is to say once skipping is what the sublog mostly does [BSF21]. A skip is shown only when the evidence for it is a majority of the behaviour, which is what R2 demands.

The price of bottom-up cut detection is computational. Per recursion step, constructing and filtering $\mapsto$ and $\mapsto^*$ costs $O(|\Sigma|^2)$, the naive FindCut costs $O(4^{|\Sigma|})$ and SplitLog costs $O(|L|)$, while each sublog's alphabet shrinks by at least one activity, so there are at most $|\Sigma|$ invocations and the overall running time is $O(|\Sigma| \cdot (4^{|\Sigma|} + |L|))$ [BSF21]. The exponential term is what makes the naive search unusable at scale, and heuristics over $\mapsto$ and $\mapsto^*$ prune the search space significantly in the implementation.

6 Evaluation and Critical Assessment

6.1 Quantitative evaluation

PIM was implemented in the UiPath Process Mining platform and compared against the three algorithms that outperformed the others in the benchmark of Augusto et al., namely the Evolutionary Tree Miner (ETM), IMf and the Split Miner (SM) [Au19; BSF21]. The accuracy figures are fitness, precision and F-score; the simplicity figures are size, control-flow complexity, structuredness and soundness.

Three objections are raised against the benchmark as a proxy for industrial work. Its logs have mostly been cleaned up in advance, which is a luxury nobody has in production. They are small by industrial standards. And the process shapes that turn up in practice are not all represented in it. To compensate, four unfiltered datasets were added: BPIC14 and BPIC17LC, both public, and two proprietary logs, Invoice and P2P [BSF21]. PIM was run

twice, both at $f = 99.5\%$: PIM30 bounds the naive FindCut to the thirty most frequent activities to respect the benchmark's four hour limit, and PIM uses fast heuristics for which no activity bound is required.

Both PIM and PIM30 discovered sound, block-structured models on all event logs, while SM returned unsound models on BPIC14 and BPIC17LC. On the benchmark logs PIM sacrifices fitness for precision, beating IMf on eleven of twelve logs, matching or beating SM on nine, and reaching the highest precision on five. F-score improves on IMf on ten of twelve; size improves on IMf on eight and on SM on four; control-flow complexity improves on IMf on ten and matches or beats SM on six. Runtime is the widest margin, PIM completing in under a second on all datasets and fastest on eleven of twelve, whereas PIM30 either finishes within a second on five of twelve logs or takes several hours [BSF21].

Condensed: ETM is accurate but takes hours, SM has the highest fitness and is not always sound, IMf is sound but complex and imprecise, and PIM is the simplest and fastest with the lowest fitness and high precision, and the only one of the four with a uniform complexity parameter.

The authors' own reading is not that PIM dominates but that it reaches a novel balance in the pareto front, with more results combining high precision and low size or control-flow complexity than IMf and SM, while the overall F-score remains comparable [BSF21]. A sensitivity study supports this rather than leaving it an artefact of parameter choice: heavier filtering for SM removed up to fifty percent of its edges, yet its size and complexity remained significantly above PIM's at $f = 99.5\%$, and heavier filtering for IMf changed accuracy and size unpredictably while also remaining above PIM's.

6.2 The trust evaluation

The empirical evaluation is the more interesting of the two, because R2 is a claim about analysts rather than metrics and no computed score can test it. Seventeen people took part, eleven of them practitioners, four researchers and two students. Each received two things. The first was a single A4 sheet per log showing as many of the most frequent trace variants as would still be legible, which came to 99% of RTFMP, 94% of Invoice and 55% of BPIC17cp. The second was the BPMN rendering of what IMf, SM and PIM had discovered from that log, laid out automatically, stripped of any mention of which algorithm produced it, and shuffled. Participants marked up each model in two colours, one for structures they understood and believed the data supported and one for structures they did not, then ranked the three and explained why [BSF21].

SM was the most preferred model for RTFMP, chosen by ten of seventeen participants, while PIM was preferred for Invoice by fifteen of seventeen and for BPIC17cp by eleven of seventeen, and of the fifty-one written motivations, thirty, or 58.8%, favoured PIM on grounds of simplicity, clarity or readability. One asymmetry in the free text carries more

weight than the counts: PIM's models were sporadically described as representing too little of the data, but never too much [BSF21]. That is the signature of an algorithm erring towards precision rather than generalisation, which is what R2 asked for.

6.3 Is PIM the best algorithm?

Read against the alternatives, the claim that PIM is best does not survive, and the paper never makes it. ETM searches genetically over process trees against all four quality dimensions at once and frequently beats IM on both precision and fitness, but the search can run for hours. SM reaches the highest raw fitness and returned unsound models on two of the four additional logs. The 2013 Inductive Miner supplies the soundness foundation and is imprecise. IMc, treated in the second half of this report, handles missing evidence most rigorously and assumes noise away. PIM does not lead on every axis. What it leads on is the combination R0 to R2 describes, and that counts as a result rather than a redefinition of the target, because the requirements were elicited from analysts before the algorithm existed. Which discovery algorithm is best has no answer until the requirements are fixed, and PIM is the best available answer to one empirically grounded set of them.

Four limitations qualify even that. The benchmark contains only prefiltered variants of real logs, and the four unfiltered logs offsetting this are few. Every participant of the user study was recruited in the Eindhoven area, a plausible source of preference bias, and seventeen is a small sample. PIM estimates its relations through frequency heuristics where IMc computes actual probabilities, so the probabilistic framing is weaker than the name suggests, a point the authors make themselves and which Section 10 develops. Least often noticed, PIM approximates behaviour whenever the underlying process is not block-structured, so its accuracy figures partly measure how block-structured the benchmark processes happen to be. That follows directly from the representational bias identified in Section 3 as the price of soundness by construction, and cannot be removed without giving up R0.

7 Transition to the Second Part

The first half of this report has established the framework side of the argument: a recursive construction whose output is sound by representation and fitting by proof, an instantiation that runs in polynomial time and rediscovers the original process from a directly-follows complete log, and a single assumption, the reading of the directly-follows graph as binary facts, whose failure on logs with deviations motivates everything that follows. It stated the three requirements PIM was built to satisfy, showed that IMf satisfies only part of them, and assessed the published evaluation.

What it did not do is open the mechanism that replaces the binary reading. Section 5 named PIM's activity relation scores and its two correction terms without defining them, and

Section 4 asserted that weighing all pairs against each other is what IMf lacks, without showing how that weighing is computed.

That is the content of the remainder. Section 8 presents the second foundational paper [LFA14b] and its activity relation lattice. Section 9 works the arithmetic through on the paper's own example, tracing (a, g) from its observed exclusive choice relation to $p_{\times}(a, g) = 0.60$, its residual sequence probability $p_{\rightarrow}(a, g) \approx 0.07$, and an aggregate root cut score of about 0.64. Section 10 gives PIM's relation scores and both correction terms in full, Section 11 computes an independent example from the same definitions, and Sections 12 and 13 close the report.

8 Foundational Paper 2: Discovering Models from Incomplete Logs

The true model and the incomplete log. The second foundational paper studies what happens when a log does not contain enough traces to display all the behaviour of the process that produced it [LFA14b]. Its authors call the resulting algorithm IMin; this report follows [BSF21] in referring to it as IMc. The situation is the mirror image of the deviation problem that closed the first half of this report: there the log showed too much, here it shows too little. The paper's example fixes a true block-structured workflow net M_E in which a left block containing a, b and c occurs before a right block containing d, e, f and g, so in the real process a can occur before g and the true relation between them is the sequence $a \rightarrow g$.

The observed log is

$$L_E = [\langle c, d, e, f, d, e, f, d, e\rangle,\ \langle b, a, d, e\rangle,\ \langle a, b, d, e, f, d, e\rangle,\ \langle c, g\rangle].$$

Reading down the four traces, a occurs in the second and third, and g occurs only in the fourth. No trace contains both. The log therefore supplies neither direct nor indirect evidence of a follow relation between them in either direction, and a miner that reads the graph as binary facts, exactly as B' does, has no choice but to classify the pair as

$$\times(a, g),$$

meaning that either a occurs or g occurs but never both. This is not the behaviour of M_E. It is an artefact of incompleteness, and it is the kind of misreading that the first half of this report said cannot be repaired once it has happened, because nothing weighs evidence.

The activity relation lattice. To reason about such cases the paper defines a lattice over the possible relations between any pair of activities, classifying the pair by which direct and transitive follows relations the log exhibits. The structure of the lattice is reproduced in simplified form in Fig. 4.

The bottom node is the exclusive choice case, reached when no path is observed in either direction; the left and right sides are the two sequence directions; the top is parallel

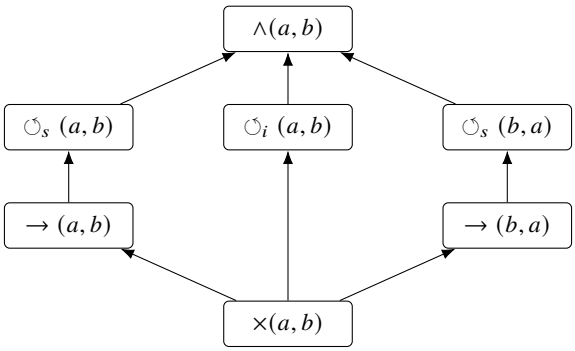


Fig. 4: Simplified activity relation lattice, after [LFA14b]. The bottom node is exclusive choice, where no follow relation is observed in either direction; the sides are the two sequence directions; the loop relations sit in the middle; and the top is parallelism, where both orders are observed. Arrows point in the direction a pair can move when further traces are added to the log.

behaviour, where both orders have been observed; and the loop cases sit between them, with $\circlearrowleft_s$ distinguishing direct entry to and exit from the redo part and $\circlearrowleft_i$ the indirect loop relation.

The direction of the arrows is what makes the lattice more than a taxonomy. Adding traces can only introduce new follow relations, never remove them, so a pair can only move upward as the log grows. The incomplete log L_E places (a, g) at the bottom node, but a more complete log could move it to $\rightarrow (a, g)$, which is where the true model puts it. Incompleteness is therefore not random error but a systematic downward bias in the lattice, which is what makes correcting for it possible.

Turning the lattice into probabilities. The correction is to stop treating the lattice position as a fact and start treating it as the most likely of several possibilities. The paper assigns each pair a probability distribution over the seven directed entries of the lattice, the five relations $\times$, $\rightarrow$, $\wedge$, $\circlearrowleft_i$ and $\circlearrowleft_s$ with the two non-commutative ones counted in both directions, with the observed relation receiving the largest share and the remaining mass distributed over the relations further traces could still reach from the observed node. The size of that share depends on how much evidence supports the pair, quantified by the average occurrence count

$$z(a, b) = \frac{|a| + |b|}{2}, \tag{1}$$

where $|a|$ denotes the number of times a occurs in the log. The probability assigned to the relation the log actually exhibits is then

$$p_{\text{observed}}(a, b) = 1 - \frac{1}{z(a, b) + 1}. \tag{2}$$

Equation 2 never reaches one, so an observed relation is never certain, and it approaches one as z grows, so a relation supported by many occurrences is trusted more than one supported by few. The quantity z measures the amount of evidence available for the pair; it does not by itself decide which relation is correct.

The remaining mass $1 - p_{\text{observed}}$ is divided equally among the relations still reachable from the observed node in the lattice.

Two rules are needed below [LFA14b]. When the log shows nothing for the pair, so that the observed relation is $\times(a, b)$, six lattice relations remain reachable and each receives $\frac{1}{6} \cdot \frac{1}{z+1}$ of the mass. When the log shows a follow relation in one direction only, the observed sequence takes $1 - \frac{1}{z+1}$, the opposite direction takes zero, and each of the four still-reachable relations receives $\frac{1}{4} \cdot \frac{1}{z+1}$.

Two consequences matter for what PIM later does with this machinery. No relation ever receives probability zero merely because it was not observed, so a pair misplaced by incompleteness can still contribute evidence for the correct operator. But a relation contradicted by an observation receives zero permanently, because under the assumption that the log contains no noise an observation can never be wrong. That second property is what makes IMc rigorous on incomplete logs and inapplicable on real-life logs.

9 Worked Example: Probabilistic Relations and Root-Cut Selection

This section traces the pair (a, g) through the whole computation and then aggregates over the candidate root cut. The calculation has two levels: first a distribution for one pair, then an average over all pairs crossing the cut [LFA14b].

Step 1: what the log actually shows. The pair (a, g) is chosen precisely because the true model and the incomplete log disagree about it.

Tabulating the occurrences across the four traces of L_E: the first contains neither a nor g; the second, $\langle b, a, d, e\rangle$, and the third, $\langle a, b, d, e, f, d, e\rangle$, contain a but not g; and the fourth, $\langle c, g\rangle$, contains g but not a.

No trace contains both, so the log holds neither direct nor indirect evidence from a to g or from g to a, and the observed lattice relation is the bottom node $\times(a, g)$. Here $\times$ is not multiplication but exclusive choice.

Step 2: the evidence factor. Counting gives $|a| = 2$ and $|g| = 1$, so by Equation 1,

$$z(a, g) = \frac{|a| + |g|}{2} = \frac{2 + 1}{2} = 1.5. \tag{3}$$

This is small, which is the arithmetic statement of the situation: the observed exclusive choice should be favoured, but not treated as certain.

Step 3: the probability of the observed relation. Because the observed relation is exclusive choice, the first rule above applies, and substituting $z(a, g) = 1.5$ into Equation 2 gives $p_{\times}(a, g) = 1 - 1/2.5 = 1 - 0.40 = 0.60$. Exclusive choice receives the highest probability because it is the relation visible in L_E, and the value is 0.60 rather than 1.00 because few occurrences support the pair. In the paper's terminology this is an artificially estimated probability expressing confidence, not a statistically learned posterior.

Step 4: the residual sequence probability. The probabilities over the seven lattice entries must sum to one, so after assigning 0.60 to exclusive choice the uncertainty left over is $1 - 0.60 = 0.40$. Starting from the bottom node, six entries remain reachable if valid traces are added: $\rightarrow (a, g)$, $\rightarrow (g, a)$, $\circlearrowleft_i (a, g)$, $\circlearrowleft_s (a, g)$, $\circlearrowleft_s (g, a)$ and $\wedge(a, g)$. The remaining 0.40 is divided equally among these six, so the share given to the sequence direction is

$$p_{\rightarrow}(a, g) = \frac{0.40}{6} = 0.0667 \approx 0.07, \tag{4}$$

and the check is immediate: $0.60 + 6 \times (0.40/6) = 1.00$.

This is the whole point of the construction. The incomplete log mainly supports exclusive choice, but it does not make a sequence impossible. A hard relation miner such as B' stops at $\times(a, g)$ and can never recover; the probabilistic method retains a small sequence value, so that the other pairs can still contribute to the cut decision.

Step 5: aggregating over the candidate cut. The calculation now moves from one pair to the complete root cut. The candidate is $\rightarrow (\{a, b, c\}, \{d, e, f, g\})$, asking whether the first block occurs before the second. The algorithm does not decide this from (a, g) alone: for a sequence cut it averages the sequence probabilities of every pair with one activity on each side, and three activities on the left with four on the right give $3 \times 4 = 12$ crossing pairs, laid out in Tab. 2.

Tab. 2: The twelve crossing sequence probabilities $p_{\rightarrow}$ for the candidate root cut $\rightarrow (\{a, b, c\}, \{d, e, f, g\})$ on L_E.

$p_{\rightarrow}$	d	e	f	g
a	0.80	0.80	0.71	0.07
b	0.80	0.80	0.71	0.07
c	0.80	0.80	0.71	0.60

Every entry comes from the same two equations. The activity counts in L_E are $|a| = |b| = |c| = 2$, $|d| = |e| = 6$, $|f| = 3$ and $|g| = 1$, and four cases cover the table. (a, d) is observed

in sequence with $z = 4$, hence $p_{\rightarrow} = 1 - 1/5 = 0.80$; (a, f) with $z = 2.5$, hence $p_{\rightarrow} \approx 0.71$; (a, g) has no follow evidence, so its share is the residual $0.40/6 \approx 0.07$ derived above; and (c, g) is directly observed in $\langle c, g \rangle$ with $z = 1.5$, hence $p_{\rightarrow} = 0.60$. The other entries repeat one of these four cases, giving six values of 0.80, three of 0.7142857, two of 0.0666667 and one of 0.60. Therefore

$$p_{\rightarrow}(\{a, b, c\}, \{d, e, f, g\}) = \frac{6(0.80) + 3(0.7142857) + 2(0.0666667) + 0.60}{12} = \frac{7.67619}{12} = 0.63968 \approx 0.64. \tag{5}$$

Adding the displayed two-decimal values instead gives $7.67/12 = 0.639$, which rounds identically.

The two weak pairs (a, g) and (b, g) contribute only two of the twelve values, and most other crossing pairs support the sequence strongly, so the sequence cut remains the highest-scoring candidate. This is exactly the weighing mechanism that Section 4 said IMf lacks: a misread pair is outvoted rather than decisive.

10 PIM's Cut Detection in Detail

10.1 From probabilities to scores

PIM extends the basic idea of IMc with principles of heuristic mining [BSF21]. For each pair (a, b) it computes a score $s_{\oplus}(a, b) \in [0, 1]$ expressing how likely it is that a and b are related by $\oplus \in \{\times, \rightarrow, \wedge, \circlearrowleft\}$ in L, aggregates these into $s_{\oplus}(\Sigma_1, \Sigma_2)$ for a partition, and selects the cut whose aggregate is maximal.

What PIM does to the lattice of Fig. 4 is the decisive change. IMc's conditions partition the possibilities cleanly, each one saying what must be true of $\mapsto$ and $\mapsto^*$ for a given operator to hold in a log with no deviations. Exclusive choice, for instance, requires that neither activity ever follow the other. PIM keeps the same five conditions but loosens every one of them from a requirement into a tendency: exclusive choice now asks only that the two rarely follow each other. Swapping absolutes for quantities is what carries the lattice across from merely incomplete logs to logs that also contain mistakes.

Definition 2 (activity relation scores, [BSF21])**.** Writing $|a|$ for the number of occurrences of a in L, and abbreviating $T = |a \mapsto b| + |b \mapsto a| + |a \mapsto^* b| + |b \mapsto^* a|$,

$$s_{\times}(a,b) = \frac{1}{2}\left(\frac{|a| - T}{|a|} + \frac{|b| - T}{|b|}\right), \tag{6}$$

$$s_{\rightarrow}(a,b) = \frac{|a \mapsto b| + |a \mapsto^* b| - (|b \mapsto a| + |b \mapsto^* a|)}{|a \mapsto b| + |a \mapsto^* b| + |b \mapsto a| + |b \mapsto^* a| + 1}, \tag{7}$$

$$s_{\wedge}(a,b) = \min\left(\frac{|a \mapsto b|}{|b \mapsto a| + 1},\ \frac{|b \mapsto a|}{|a \mapsto b| + 1}\right), \tag{8}$$

$$s_{\circlearrowleft_s}(a,b) = \min\left(\frac{|a \mapsto b|}{|b \mapsto^* a| + 1},\ \frac{|b \mapsto^* a|}{|a \mapsto b| + 1}\right), \tag{9}$$

$$s_{\circlearrowleft_i}(a,b) = \min\left(\frac{|a \mapsto^* b|}{|b \mapsto^* a| + 1},\ \frac{|b \mapsto^* a|}{|a \mapsto^* b| + 1}\right). \tag{10}$$

Each formula encodes one fuzzy condition. The paper illustrates them on its own running example

$$L_0 = [\ \langle a,b,c,e\rangle,\ \langle a,c,b,f\rangle,\ \langle a,b,c,d,c,b,e\rangle^2,\ \langle a,c,b,d,b,c,f\rangle,$$
$$\langle a,c,b,d,b,c,d,b,c,f\rangle,\ \langle a,g\rangle^9,\ \langle a,g,c,g\rangle\ ].$$

For $\times$, how often a and b occur alone is compared with how often they occur together: if only one of them occurs in any trace then $T = 0$ and $s_{\times}(a,b) = 1$, and otherwise the subtracted term grows and the score approaches zero; in L_0, $s_{\times}(b,g) = 1$. For $\rightarrow$, PIM adopts the directional dependency heuristic of the Structured Heuristic Miner, normalising the difference and truncating negative values to zero; $s_{\rightarrow}(a,g) = 0.92$. For $\wedge$, the more equal $|a \mapsto b|$ and $|b \mapsto a|$ are, the more evidence there is for a parallel relation; $s_{\wedge}(b,c) = \min(5/7, 6/6) = 0.71$. For $\circlearrowleft$ two scores are computed, one for entry to and exit from the redo part and one for the indirect loop relation, because a single score cannot partition the activities.

In contrast to IMc these scores are not probabilities: they do not sum to one over the operators, and the authors describe them plainly as heuristics [BSF21], which is the difference the first half flagged. What they retain is the discriminating property the hard conditions had: if L is directly-follows complete and $\oplus$ mostly holds between a and b, then $s_{\oplus}(a,b) > s_{\otimes}(a,b)$ for every other operator. Distinguishing $\wedge$ from the two loop relations is the one case the pairwise scores cannot settle, since both produce edges in both directions; that requires counting repetitions during aggregation.

10.2 Aggregation and the standard deviation correction

For a log without deviations, IMc defines the aggregate as the plain average $\mu(S(c))$ of all activity relation scores $S(c)$ crossing the cut c. Deviations may give a few pairs a biased

score, producing a higher average for the wrong operator even though the correct one scores higher on most pairs [BSF21], so PIM introduces a correction that lowers the aggregate when there is evidence of such a bias. For $\times$ and $\rightarrow$ that correction is the standard deviation, because the two scores are complementary: $s_{\oplus}(a, b)$ is low exactly when some other relation holds between a and b, so a high average built on a few outlying pairs signals a wrong cut rather than a strong one, and the spread detects it.

Definition 3 (aggregate score for $\times$ and $\rightarrow$, [BSF21])**.** Let $\Sigma_1 \cup \Sigma_2 = \Sigma$ and $\oplus \in \{\times, \rightarrow\}$. Then $c = (\oplus, \Sigma_1, \Sigma_2)$ is an $\oplus$-cut with score

$$s_{\oplus}(\Sigma_1, \Sigma_2) = \mu(S(c)) - \sigma(S(c)). \tag{11}$$

The paper's own illustration makes the effect concrete. Two cuts with score bags $S(c_1) = [0.6, 0.6, 0.7]$ and $S(c_2) = [0.9, 0.8, 0.4]$ give $s_{\times} = 0.63 - 0.05 = 0.58$ and $s_{\rightarrow} = 0.7 - 0.21 = 0.48$, so the first has the higher aggregate even though its raw mean is lower.

On the running example L_0 the same correction reverses a real decision. Consider $c_1 = (\times, \{a, b, c, d, e, f\}, \{g\})$ and $c_2 = (\rightarrow, \{a\}, \{b, c, d, e, f, g\})$. The raw means are $\mu(c_1) = 0.86 > \mu(c_2) = 0.79$, because g has strong exclusive choice relations to b, c, d, e and f. But $s_{\times}(a, g) = 0.16$ shows that some other relation holds between a and g and that $\mu(c_1)$ is falsely biased towards $\times$, which surfaces as $\sigma(c_1) = 0.29$ against $\sigma(c_2) = 0.13$. The paper reports the corrected scores as 0.54 and 0.67, making c_2 the more likely cut [BSF21]. For c_2 that is consistent with the figures given, since any μ and σ rounding to 0.79 and 0.13 leave $\mu - \sigma$ in $[0.650, 0.670]$. For c_1 it is not: the same test puts $\mu - \sigma$ in $[0.560, 0.580]$, so no rounding of 0.86 and 0.29 yields 0.54. As with $r(L_5)$ below, the reported value and the stated inputs do not reconcile, though the cut PIM prefers is unaffected.

10.3 The repetition ratio correction

For $\wedge$ and $\circlearrowleft$ a different correction is needed, because the pairwise scores cannot separate parallel from loop behaviour; distinguishing them requires explicitly counting repetitions. If L has no repetition then each trace contains each $a \in \Sigma$ at most once and $\|L\| \leq |L| \cdot |\Sigma|$. The paper's illustrations are $L_1 = [\langle a, b\rangle, \langle b, a\rangle]$ with $\|L_1\| = 4 \leq 2 \cdot 2$, and $L_2 = [\langle a, b\rangle, \langle b, a, b\rangle]$ with $5 \not\leq 4$. Defining

$$r(L) = \frac{|L|}{\|L\|/|\Sigma|}, \tag{12}$$

we get $r(L_1) = 1$ and $r(L_2) = 2/(5/2) = 0.8$, so $r(L) < 1$ indicates the presence of a loop [BSF21].

Definition 4 (aggregate scores for $\wedge$ and $\circlearrowleft$, [BSF21])**.** Let $\Sigma_1 \cup \Sigma_2 = \Sigma$. Then

$$s_{\wedge}(\Sigma_1, \Sigma_2) = \mu(S(c)) \cdot \min(r(L), 1), \tag{13}$$

$$s_{\circlearrowleft}(\Sigma_1, \Sigma_2, S_2, E_2) = \mu(S(c)) + \big(\mu(S(c)) \cdot (1 - \min(r(L), 1))\big). \tag{14}$$

The first factor reduces the parallel score in the presence of loops. Since $r(L) \approx 1$ indicates only the absence of a loop and not the presence of parallelism, the loop score is instead boosted by the inverse $1 - \min(r(L), 1)$. The running example again shows the correction reversing a decision. On the sublog $L_5 = [\langle b, c\rangle, \langle c, b\rangle, \langle b, c, d, c, b\rangle^2, \langle c, b, d, b, c\rangle, \langle c, b, d, b, c, d, b, c\rangle]$ the candidates are $c_6 = (\wedge, \{b\}, \{c, d\})$ and $c_7 = (\circlearrowleft, \{b, c\}, \{d\})$. Without correction $\mu(S(c_6)) = 1.0$ and $\mu(S(c_7)) = 0.79$, so the parallel cut would win. With $r(L_5) = 0.75$ the correction factors give

$$s_{\wedge}(\{b\}, \{c, d\}) = 1.0 \times 0.75 = 0.75, \tag{15}$$

$$s_{\circlearrowleft}(\{b, c\}, \{d\}) = 0.79 \times (2 - 0.75) = 0.79 \times 1.25 = 0.9875 \approx 0.99, \tag{16}$$

and PIM returns c_7 [BSF21]. Recomputing $r(L_5)$ from Equation 12 on L_5 as printed gives $6/(27/3) \approx 0.67$ rather than the 0.75 the paper reports, so the two are not consistent with each other. The discrepancy does not affect the argument: at 0.67 the corrected scores become 0.67 and 1.05, and the loop cut still wins. Tab. 3 collects both reversals side by side. In each case the raw average selects the wrong cut and the correction reverses the decision, which is the strongest single argument the paper makes for its design.

Tab. 3: Raw and corrected cut scores on the running example L_0 of [BSF21]. In both cases the raw average selects the wrong cut and the correction reverses the decision.

Candidate cut	Operator	$\mu(S(c))$	Correction	Final score
$c_1 = (\{a, \ldots, f\}, \{g\})$	$\times$	0.86	$\sigma = 0.29$	0.54
$c_2 = (\{a\}, \{b, \ldots, g\})$	$\rightarrow$	0.79	$\sigma = 0.13$	**0.67**
$c_6 = (\{b\}, \{c, d\})$	$\wedge$	1.00	$\times r(L_5) = 0.75$	0.75
$c_7 = (\{b, c\}, \{d\})$	$\circlearrowleft$	0.79	$\times (2 - r(L_5)) = 1.25$	**0.99**

11 Own Worked Example: A Small Approval Process

The example of Section 9 follows the paper's own numbers. This section applies the same two equations to a log constructed for this report.

Setup. Consider a small approval process with four activities: a (register request), b (check documents), c (reject immediately) and d (pay out). Three cases were recorded: $L_{\text{own}} = [\langle a, b, d\rangle, \langle a, b, d\rangle, \langle a, c\rangle]$. Two requests were checked and paid, one was rejected at registration, and the activity counts are $|a| = 3$, $|b| = 2$, $|c| = 1$ and $|d| = 2$. Checking documents and rejecting immediately are genuine alternatives here, so exclusive choice is the correct relation for (b, c) and, unlike in the paper's example, the log is not misleading about it. The instructive part is what the single observed pair (a, c) does to the competing cuts.

Pairwise values. Applying Equations 1 and 2 to each pair gives $p_{\rightarrow}(a, b) = 0.7143$ with $z = 2.5$, $p_{\rightarrow}(a, c) = 0.6667$ with $z = 2$, $p_{\rightarrow}(a, d) = 0.7143$ with $z = 2.5$ and $p_{\rightarrow}(b, d) = 0.6667$ with $z = 2$, while the two pairs with nothing observed, (b, c) and (c, d), get $p_{\times} = 0.60$ with $z = 1.5$. Note that $p_{\times} = 0$ for every pair where a follow relation is observed in either direction.

Two entries deserve comment. For (b, c) and (c, d) nothing is observed in either direction, so the first rule applies and $p_{\times} = 1 - 1/2.5 = 0.60$ with a residual sequence share of $0.40/6 \approx 0.0667$. For (a, d) the activities never occur adjacently, but d follows a indirectly in the first two traces, so the second rule applies with $z = 2.5$ and $p_{\rightarrow} = 0.7143$.

Three candidate root cuts. The correct root cut is $\rightarrow (\{a\}, \{b, c, d\})$, since every trace begins with a. Averaging its three crossing sequence probabilities,

$$p_{\rightarrow}(\{a\}, \{b, c, d\}) = \frac{0.7143 + 0.6667 + 0.7143}{3} = \frac{2.0953}{3} = 0.6984 \approx 0.70. \tag{17}$$

A competing exclusive choice cut $\times(\{a, b, d\}, \{c\})$ looks attractive, because c never co-occurs with b or d. Its three crossing pairs give

$$p_{\times}(\{a, b, d\}, \{c\}) = \frac{0 + 0.60 + 0.60}{3} = \frac{1.20}{3} = 0.40. \tag{18}$$

The single pair (a, c), directly observed in $\langle a, c\rangle$ and therefore scoring $p_{\times} = 0$, drags the average down far enough to lose: one observation contradicting exclusive choice outweighs two supporting it. A third candidate, $\rightarrow (\{a, b\}, \{c, d\})$, splits the alternatives across the cut and its four crossing pairs give

$$p_{\rightarrow}(\{a, b\}, \{c, d\}) = \frac{0.6667 + 0.7143 + 0.0667 + 0.6667}{4} = 0.5286 \approx 0.53. \tag{19}$$

Here the weak pair (b, c), contributing only 0.0667, costs the candidate the decision.

What the example shows. The correct cut wins at 0.70 against 0.53 and 0.40, and the two losers lose for opposite reasons: the exclusive choice cut is defeated by one contradicting observation, which a hard relation miner could not have expressed as a matter of degree, and the misplaced sequence cut by one unsupported pair whose residual probability is small but not zero. The example also exposes the limitation that motivates PIM. Nothing here distinguishes an unobserved pair that is genuinely exclusive from one merely missing, and nothing discounts an observation that is a data-entry error rather than real behaviour: if $\langle a, c\rangle$ were a deviation, the arithmetic would be unchanged and the exclusive choice cut would still be penalised by it. Correcting that requires the frequency-based scores and correction terms of Section 10.

12 Discussion

12.1 How the three papers connect

The three papers form a chain in which each supplies what its predecessor lacks. The 2013 paper contributes the sound recursive framework [LFA13] and none of the numbers here, because it computes no frequencies at all. The 2014 paper contributes the lattice and the idea of assigning non-zero probability to relations that were not directly observed [LFA14b]; its weakness is its assumption, since the problem is treated as incompleteness and never as noise. The 2021 paper contributes the algorithm usable in practice [BSF21], adding percentile filtering and the two correction terms so that deviations can be handled. PIM is best understood not as a new algorithm but as a bridge between theoretical soundness and practical trust.

12.2 Strengths and limits

PIM's main strength is that it produces block-structured models whose structures are supported by frequency evidence, and that it gives analysts a parameter to control model complexity, together addressing the industrial concern that models be understandable, trustworthy and still fit the observed behaviour. The trade-offs are equally real: filtering infrequent behaviour may discard behaviour that is rare but genuine, favouring stable structures sacrifices fitness, and because the scoring formulas are heuristic rather than probabilistic the results should be read as evidence-based model choices rather than exact statistical statements. Two further limitations belong to the evaluation rather than the algorithm, as Section 6 argued: the user study rests on seventeen participants recruited in a single region, and the accuracy figures partly reflect how block-structured the benchmark processes happen to be.

13 Conclusion

The 2013 framework settled a problem open for two decades by changing what had to be proved. Restricting the output to block-structured process trees makes soundness a property of the representation, leaving only fitness and termination, and B' makes the framework practical in polynomial time while rediscovering the original process from any directly-follows complete log. That strength is also the weakness: reading the graph as binary facts is what makes the cut conditions checkable and the proofs possible, and also what lets a single deviating trace turn a strict sequence into a permanent parallel block. The 2014 incomplete-log paper supplies the missing mechanism by softening hard relations into probabilities, so that a misread pair is outvoted rather than decisive: (a, g) appears to be an exclusive choice and receives $p_{\times} = 0.60$, yet the sequence remains possible at $p_{\rightarrow} \approx 0.07$, and averaging over the twelve crossing pairs still awards the correct root cut about 0.64.

PIM carries that mechanism onto real data without touching the framework. The recursion, the four cut types and soundness by construction are inherited unchanged and satisfy R0 for free; percentile filtering ahead of cut detection satisfies R1; and cut selection is rebuilt on frequency-based scores with a standard deviation penalty and a repetition correction, so structures appear only when the evidence is strong and consistent, satisfying R2. The question PIM asks is therefore not which relation is still plausible if the log is incomplete, but which is frequent and stable enough to be trusted in a noisy log. The result does not lead on all dimensions. It reaches a region of the trade-off between them that the other algorithms do not occupy, inside a framework that still guarantees soundness, and analysts preferred it in a controlled comparison. A question the three papers leave open follows from the framework rather than the algorithm: since it admits any operator with a monotone language join function, and since PIM's accuracy partly reflects how block-structured the process is, extending the operator set rather than the scoring is a direction none of them explores.

References

[Aa16] van der Aalst, W. M. P.: Process Mining: Data Science in Action. Springer, 2016.

[Aa22] van der Aalst, W. M. P.: Foundations of Process Discovery. In: Process Mining Handbook. Vol. 448, LNBIP, Springer, pp. 37–75, 2022.

[Au19] Augusto, A. et al.: Automated Discovery of Process Models from Event Logs: Review and Benchmark. IEEE Transactions on Knowledge and Data Engineering 31 (4), pp. 686–705, 2019.

[BDA14] Buijs, J. C. A. M.; van Dongen, B. F.; van der Aalst, W. M. P.: Quality Dimensions in Process Discovery: The Importance of Fitness, Precision, Generalization and Simplicity. International Journal of Cooperative Information Systems 23 (1), 2014.

[BSF21] Brons, D.; Scheepens, R.; Fahland, D.: Striking a New Balance in Accuracy and Simplicity with the Probabilistic Inductive Miner. In: Proceedings of the 3rd International Conference on Process Mining (ICPM 2021). IEEE, pp. 32–39, 2021.

[LFA13] Leemans, S. J. J.; Fahland, D.; van der Aalst, W. M. P.: Discovering Block-Structured Process Models from Event Logs: A Constructive Approach. In: Application and Theory of Petri Nets and Concurrency (Petri Nets 2013). Vol. 7927. LNCS, Springer, pp. 311–329, 2013.

[LFA14a] Leemans, S. J. J.; Fahland, D.; van der Aalst, W. M. P.: Discovering Block-Structured Process Models from Event Logs Containing Infrequent Behaviour. In: Business Process Management Workshops (BPM 2013). Vol. 171. LNBIP, Springer, pp. 66–78, 2014.

[LFA14b] Leemans, S. J. J.; Fahland, D.; van der Aalst, W. M. P.: Discovering Block-Structured Process Models from Incomplete Event Logs. In: Application and Theory of Petri Nets and Concurrency (Petri Nets 2014). Vol. 8489. LNCS, Springer, pp. 91–110, 2014.